Introduzione a Python

Indice

Introduzione

Caratteristiche principali del linguaggio di programmazione

Variabili e tipi di dato

Ingresso e uscita

Operatori

Istruzioni condizionali

Cicli

Liste

Funzioni

Un algoritmo completo: il valore massimo in un array

Operatori bit a bit

Python e C++ a confronto

Introduzione

Introduzione: un secondo linguaggio di programmazione

- Nel capitolo «Introduzione a C++» abbiamo tradotto lo pseudocodice dei nostri algoritmi in un linguaggio di programmazione **compilato**
- In questo capitolo introduciamo un secondo linguaggio di programmazione, **Python**
- Python è **interpretato**, ha una sintassi più leggera ed è molto diffuso sia nella didattica sia nel calcolo scientifico

Il percorso del capitolo

Il capitolo segue lo stesso percorso del capitolo «Introduzione a C++», in modo che i due linguaggi possano essere confrontati sezione per sezione:

- le caratteristiche principali del linguaggio e la struttura di un programma
- variabili, tipi di dato, ingresso e uscita
- operatori, istruzioni condizionali e cicli
- le liste, che svolgono il ruolo degli array di C++
- le funzioni, i loro parametri e i loro valori di ritorno
- un algoritmo completo, la ricerca del valore massimo in un array, scritto in pseudocodice e in Python
- gli operatori bit a bit e una tabella riassuntiva che confronta Python con C++

Codice eseguito

Ogni frammento di codice del capitolo è stato eseguito
con l'interprete `python3` (`Python 3.14`).

Gli output riportati negli esempi
sono quelli stampati dai programmi.

Indici: pseudocodice e Python

- Nello pseudocodice le componenti di un array $a[1..n]$ sono indicizzate da 1 a n
- In Python, come in C++, gli elementi di una lista di dimensione n sono indicizzati da 0 a $n-1$
- La componente $a[i]$ dello pseudocodice corrisponde all'elemento `a[i-1]` del codice Python

Riepilogo

- Python è un secondo linguaggio di programmazione, interpretato e con una sintassi leggera
- Il capitolo segue lo stesso percorso del capitolo su C++, per confrontare i due linguaggi
- Gli indici dello pseudocodice vanno da 1 a n, quelli di Python da 0 a $n-1$: $a[i]$ diventa `a[i-1]`

Caratteristiche principali del linguaggio di programmazione

Introduzione: Caratteristiche principali del linguaggio

- In questa sezione vediamo che cosa distingue Python da C++: interprete, tipizzazione dinamica, indentazione
- Vediamo come si scrivono i commenti e com'è strutturato un programma
- Queste regole valgono per ogni riga di codice che scriveremo nel resto del capitolo

Caratteristiche principali (1/2)

- Python è un linguaggio **interpretato**: i programmi sono eseguiti istruzione per istruzione dall'**interprete** Python, senza una fase di compilazione separata
  - questo rende lo sviluppo e il debug rapidi e interattivi
- Python usa la **tipizzazione dinamica**: le variabili non devono essere dichiarate con un tipo di dato
  - il tipo di dato è determinato automaticamente, durante l'esecuzione, dal valore assegnato
- Python supporta le **funzioni**, che permettono un approccio strutturato e modulare alla programmazione
  - le funzioni consentono di riutilizzare il codice e di organizzarlo meglio

Caratteristiche principali (2/2)

- **L'indentazione è obbligatoria** in Python: la struttura a blocchi del codice (cicli, istruzioni condizionali, funzioni) è definita dal livello di indentazione, non dalle parentesi graffe
  - l'indentazione standard è di 4 spazi
- Un commento su una sola riga inizia con il simbolo riservato "#"
- Non esiste un simbolo dedicato ai commenti su più righe: un testo racchiuso fra tripli apici """ (oppure ''') è una stringa che il programma non usa, e viene comunemente usato come commento su più righe
- La stessa stringa, scritta come prima riga di una funzione, è la sua **stringa di documentazione** (`docstring`)

Esempio: commenti in Python

Questo è un esempio di commento su una sola riga:

```
1 # Questa è una riga di commento
```

Questo è un esempio di commento su più righe:

```
1 """
2 Questo è un
3 commento su più righe
4 """
```

La struttura di un programma

- Un programma Python è un file di testo (con estensione `.py`) che l'interprete esegue dalla prima riga all'ultima
- A differenza di C++, **non è richiesta una funzione** `main`: le istruzioni scritte al livello più esterno del file sono eseguite in ordine

Esempio: lo scheletro di un programma

```
1 # Qui si scrivono le istruzioni del programma
2 print("Ciao, mondo!")
```

Se il codice sorgente è salvato nel file `hello.py`, lo si esegue con il comando

```
python3 hello.py
```

e l'output è

```
Ciao, mondo!
```

Negli esempi seguenti, a meno che non contengano un programma completo, i frammenti di codice sono scritti al livello più esterno di un file ed eseguiti in questo modo.

Blocchi di istruzioni

- Un **blocco** di istruzioni (il corpo di un'istruzione condizionale, di un ciclo o di una funzione) è introdotto dal simbolo " :" alla fine della riga precedente
- Tutte le sue istruzioni sono indentate dello stesso numero di spazi
- Il blocco termina alla prima istruzione indentata di meno

Esempio: blocchi definiti dall'indentazione

```
1 x = 7
2 if x > 0:
3     print("positivo")              # Dentro il blocco dell'if
4     print("ancora dentro")         # Dentro il blocco dell'if
5 print("fuori dal blocco")         # Eseguita in ogni caso
```

Output:

```
positivo
ancora dentro
fuori dal blocco
```

In C++ lo stesso blocco sarebbe racchiuso fra parentesi graffe `{}`; in Python un'indentazione sbagliata cambia il significato del programma o produce un `IndentationError`.

Niente punti e virgola, niente parentesi graffe

In Python le istruzioni non terminano con “;”,
e i blocchi sono delimitati dall’indentazione (4 spazi)
invece che dalle parentesi graffe.

Riepilogo

Il linguaggio:

- Interpretato, con tipizzazione dinamica e funzioni
- Commenti con #; tripli apici `"""` per i commenti su più righe e le docstring

La struttura di un programma:

- Un file `.py` eseguito dalla prima riga all'ultima, lanciato con `python3`; nessuna funzione `main`
- Blocchi introdotti da ":" e delimitati dall'indentazione (4 spazi)

Variabili e tipi di dato

Introduzione: Variabili e tipi di dato

- Il codice sorgente opera su **variabili**, e ogni valore ha un **tipo di dato**
- In questa sezione vediamo i quattro tipi di dato fondamentali di Python e come si creano le variabili
- La differenza principale con C++: niente dichiarazioni, il tipo di dato non è scritto nel codice

I quattro tipi di dato fondamentali (1/2)

Le informazioni sono classificate in diversi **tipi di dato**. I quattro tipi di dato fondamentali sono:

1. Il tipo di dato "`int`" rappresenta un numero intero, cioè un numero scritto senza la virgola decimale
   - comprende i numeri interi positivi e negativi, e lo zero
   - a differenza di C++, gli interi di Python **non hanno un valore massimo**: possono avere tutte le cifre che servono

2. Il tipo di dato "`bool`" rappresenta un valore booleano: `True` o `False` (si noti l'iniziale maiuscola)

I quattro tipi di dato fondamentali (2/2)

3. Il tipo di dato "`float`" rappresenta un numero in virgola mobile, che comprende i numeri scritti con la virgola decimale
   - un `float` di Python ha la stessa precisione (doppia) di un `double` di C++

4. Il tipo di dato "`str`" rappresenta una stringa di caratteri, racchiusa fra apici singoli o doppi (ad esempio `'ciao'`, `"mondo"`)
   - non esiste un tipo di dato separato per un singolo carattere: un carattere è una stringa di lunghezza 1

Variabili

- Una variabile funziona come un contenitore che memorizza un valore. Ogni variabile ha un nome univoco
- Alle variabili si assegnano valori con l'operatore di assegnamento "=". Quando si assegna un nuovo valore, il valore precedente viene sovrascritto
- **Non serve alcuna dichiarazione** prima dell'uso: una variabile viene creata nel momento in cui le si assegna un valore
- La funzione `type()` restituisce il tipo di dato di un valore

Le variabili si creano con l'assegnamento

Le variabili sono create tramite assegnamento;
non c'è una fase di dichiarazione separata,
e il tipo di dato non è scritto nel codice.

Esempio: assegnamenti di variabili (1/2)

```
1  age = 25                          # Variabile intera
2  is_raining = False                # Variabile booleana
3  small_number = 3.14               # Variabile in virgola
       mobile
4  large_number = 123456.789         # Variabile in virgola
       mobile
5  character = 'a'                   # Variabile stringa (un solo
        carattere)
6  name = "Alice"                    # Variabile stringa
7
8  age = 26  # Il valore precedente 25 viene sovrascritto
9  print(age, is_raining, small_number, large_number,
       character, name)
10 print(type(age), type(is_raining), type(small_number),
       type(name))
```

Esempio: assegnamenti di variabili (2/2)

Output:

```
26 False 3.14 123456.789 a Alice
<class 'int'> <class 'bool'> <class 'float'> <class 'str'>
```

- Si confronti con l'esempio sulle stampe sullo schermo della sezione successiva e con l'esempio corrispondente del capitolo «Introduzione a C++»
- Il valore booleano è visualizzato come `False`
- Il numero `123456.789` è visualizzato con tutte le sue cifre

Esempio: tipizzazione dinamica e interi grandi

```
1 x = 5
2 print(x, type(x))
3 x = "cinque"  # Lo stesso nome ora si riferisce a una
      stringa
4 print(x, type(x))
5
6 print(2 ** 100)  # Un intero con 31 cifre
```

Output:

```
5 <class 'int'>
cinque <class 'str'>
1267650600228229401496703205376
```

L'operatore ** è l'elevamento a potenza (si vedano gli operatori aritmetici). In C++ il valore 2^{100} non entra in una variabile di tipo `int`.

Riepilogo

Tipi di dato:

- `int` (nessun valore massimo), `bool` (`True`, `False`), `float` (precisione doppia), `str` (un carattere è una stringa di lunghezza 1)

Variabili:

- Create tramite assegnamento con "=", senza dichiarazione
- Lo stesso nome può riferirsi in seguito a un valore di un altro tipo di dato
- `type()` restituisce il tipo di dato di un valore

Ingresso e uscita

Introduzione: Ingresso e uscita

- Un programma comunica con l'utente: visualizza i risultati e legge i dati
- In questa sezione vediamo la funzione `print()` per l'uscita e la funzione `input()` per l'ingresso
- Vediamo anche le f-string, che combinano testo e valori in modo compatto

La funzione `print()`

- La funzione `print()` visualizza un output sullo schermo
- Più valori possono essere stampati in una sola chiamata separandoli con virgole: sono visualizzati separati da uno spazio, e la riga termina con un a capo (lo stesso effetto di `endl` in C++)
- I parametri opzionali `sep` e `end` cambiano il separatore fra i valori e il testo scritto alla fine
  - `print(x, end=" ")` scrive uno spazio invece dell'a capo, così la `print` successiva continua sulla stessa riga

Le f-string

- Le stringhe e i valori delle variabili possono essere combinati usando le **f-string** (stringhe formattate), precedute dalla lettera `f`
- Dentro le f-string i nomi delle variabili (e, più in generale, le espressioni) si scrivono fra parentesi graffe `{}`
- Dentro le parentesi graffe di una f-string, il formato `:.kf` visualizza un numero in virgola mobile con esattamente k cifre dopo la virgola decimale (lo stesso effetto di `fixed` e `setprecision(k)` in C++)

Esempio: stampe sullo schermo (1/2)

```
1 n = 42
2 print("Il valore della variabile è", n)
3 print(f"Il valore della variabile è {n}")  # Versione
    con f-string
4 print(f"n = {n}, doppio = {n * 2}")        #
    Espressione dentro {}
5
6 print("a", "b", "c", sep="-")   # Separatore "-" invece
     dello spazio
7 print("Niente a capo", end=" ") # Uno spazio invece
    dell'a capo
8 print("dopo end")
9 print(f"{10 / 3:.2f}")          # Due cifre dopo la
    virgola decimale
```

Esempio: stampe sullo schermo (2/2)

Output:

```
Il valore della variabile è 42
Il valore della variabile è 42
n = 42, doppio = 84
a-b-c
Niente a capo dopo end
3.33
```

La funzione `input()`

- La funzione `input()` visualizza un messaggio, attende che l'utente scriva una riga di testo e la restituisce
- Restituisce sempre una `str`
- Per ottenere un numero, il risultato deve essere convertito con `int()` o `float()`

Esempio: ingresso da tastiera

```python
name = input("Inserisci il tuo nome: ")          # Legge una stringa
n = int(input("Inserisci un intero: "))          # Legge e converte in int
x = float(input("Inserisci un numero reale: "))  # Legge e converte in float
print(f"Ciao {name}, n = {n}, x = {x}")
```

Esempio di esecuzione (l'utente scrive `Ada`, `7` e `2.5`):

```
Inserisci il tuo nome: Ada
Inserisci un intero: 7
Inserisci un numero reale: 2.5
Ciao Ada, n = 7, x = 2.5
```

Riepilogo

Uscita:

- `print()` scrive i valori separati da uno spazio e termina la riga
- I parametri `sep` e `end` cambiano il separatore e la fine della riga
- f-string: `f"...{espressione}..."`, con `:.kf` per k cifre decimali

Ingresso:

- `input()` restituisce sempre una `str`
- I numeri si ottengono con `int()` o `float()`

Operatori

Introduzione: Operatori

- Gli operatori sono simboli che eseguono operazioni su variabili e valori
- Come in C++, sono classificati, secondo la loro funzione, in operatori **aritmetici**, di **assegnamento**, **relazionali** e **logici**
- Python **non ha operatori di incremento e decremento**
- Facciamo attenzione alle differenze con C++, in particolare per la divisione

Operatori aritmetici

Operatori aritmetici, usati per eseguire le operazioni matematiche di base:

- + (Addizione) $\rightsquigarrow$ Somma due valori
- - (Sottrazione) $\rightsquigarrow$ Sottrae un valore da un altro
- * (Moltiplicazione) $\rightsquigarrow$ Moltiplica due valori
- / (Divisione) $\rightsquigarrow$ Divide un valore per un altro (restituisce sempre un `float`)
- // (Divisione intera) $\rightsquigarrow$ Restituisce il quoziente intero (divisione con arrotondamento per difetto)
- % (Resto) $\rightsquigarrow$ Restituisce il resto di una divisione
- ** (Elevamento a potenza) $\rightsquigarrow$ Eleva un valore a una potenza

Esempio: operatori aritmetici (1/2)

```
1  a = 10
2  b = 3
3
4  print("Addizione:", a + b)
5  print("Sottrazione:", a - b)
6  print("Moltiplicazione:", a * b)
7  print("Divisione:", a / b)            # Sempre un float
8  print("Divisione intera:", a // b)
9  print("Resto:", a % b)
10 print("Elevamento a potenza:", a ** b)
```

Esempio: operatori aritmetici (2/2)

Output:

```
Addizione: 13
Sottrazione: 7
Moltiplicazione: 30
Divisione: 3.3333333333333335
Divisione intera: 3
Resto: 1
Elevamento a potenza: 1000
```

L'ultima cifra `5` della divisione non è un errore del programma: 10/3 ha infinite cifre decimali, e viene visualizzato il `float` più vicino.

La divisione: Python e C++

- L'operatore di divisione `/` restituisce sempre un `float` in Python 3, anche quando divide due interi
- Per ottenere un risultato intero si usa l'operatore di divisione intera `//`
- È la **convenzione opposta** rispetto a C++, dove `10 / 3` è la divisione intera

Divisione per difetto con i numeri negativi

- L'operatore // calcola $\lfloor a/b \rfloor$, arrotondando per difetto
- Per operandi non negativi coincide con la divisione intera di C++
- Per un quoziente negativo non coincide, perché C++ tronca verso lo zero:
  - in Python `-7 // 2` vale `-4` e `-7 % 2` vale `1`
  - in C++ `-7 / 2` vale `-3` e `-7 % 2` vale `-1`
- In entrambi i linguaggi vale l'identità $a = b \cdot q + r$ fra quoziente q e resto r

Conversione di tipo

- Un valore è convertito in un altro tipo di dato (**conversione di tipo**) con le funzioni `int()`, `float()` e `str()`
- Esempi:
  - `int(3.9)` vale 3 (la parte decimale è eliminata)
  - `float(7)` vale `7.0`
  - `int("12")` è l'intero `12`

Esempio: divisione intera e in virgola mobile

```python
a = 10
b = 3

print(a / b)                # Divisione in virgola mobile
print(a // b)               # Divisione intera
print(f"{a / b:.2f}")      # Due cifre decimali
print(-7 // 2, -7 % 2)      # Divisione per difetto con quoziente negativo
print(int(-7 / 2))          # Troncamento verso zero, come in C++
print(int(3.9), float(7), int("12") + 1)
```

Output:

```
3.3333333333333335
3
3.33
-4 1
-3
3 7.0 13
```

Operatori di assegnamento (1/2)

Operatori di assegnamento, usati per assegnare valori alle variabili:

- = (Assegnamento) $\rightsquigarrow$ Assegna un valore a una variabile
- += (Assegnamento con addizione) $\rightsquigarrow$ Somma un valore e assegna il risultato
- -= (Assegnamento con sottrazione) $\rightsquigarrow$ Sottrae un valore e assegna il risultato
- *= (Assegnamento con moltiplicazione) $\rightsquigarrow$ Moltiplica e assegna il risultato

Operatori di assegnamento (2/2)

- /= (Assegnamento con divisione) $\rightsquigarrow$ Divide e assegna il risultato (un `float`)
- //= (Assegnamento con divisione intera) $\rightsquigarrow$ Divide per difetto e assegna il risultato
- %= (Assegnamento con resto) $\rightsquigarrow$ Calcola il resto e assegna il risultato
- **= (Assegnamento con elevamento a potenza) $\rightsquigarrow$ Eleva a una potenza e assegna il risultato

Esempio: operatori di assegnamento (1/3)

```
1  x = 10  # Assegnamento
2  print("Valore iniziale:", x)
3
4  x += 5   # Equivalente a: x = x + 5
5  print("Dopo += 5:", x)
6
7  x -= 3   # Equivalente a: x = x - 3
8  print("Dopo -= 3:", x)
9
10 x *= 2   # Equivalente a: x = x * 2
11 print("Dopo *= 2:", x)
```

Esempio: operatori di assegnamento (2/3)

```
12
13 x //= 4  # Equivalente a: x = x // 4
14 print("Dopo //= 4:", x)
15
16 x %= 4   # Equivalente a: x = x % 4
17 print("Dopo %= 4:", x)
18
19 x **= 3  # Equivalente a: x = x ** 3
20 print("Dopo **= 3:", x)
21
22 x /= 4   # Equivalente a: x = x / 4 (il risultato è un
       float)
23 print("Dopo /= 4:", x)
```

Esempio: operatori di assegnamento (3/3)

Output:

```
Valore iniziale: 10
Dopo += 5: 15
Dopo -= 3: 12
Dopo *= 2: 24
Dopo //= 4: 6
Dopo %= 4: 2
Dopo **= 3: 8
Dopo /= 4: 2.0
```

Incremento e decremento

- Python non ha gli operatori ++ e - -
- Una variabile si incrementa con `x += 1` e si decrementa con `x -= 1`
- L'istruzione `x++` è un errore di sintassi
- L'espressione `++x` è accettata, ma non incrementa `x`: applica due volte il più unario, quindi il suo valore è `x` stesso

Esempio: incremento e decremento

```
1 x = 5
2 x += 1            # Incremento
3 print("Dopo x += 1:", x)
4 x -= 1            # Decremento
5 print("Dopo x -= 1:", x)
6 print("++x vale", ++x, "e x vale ancora", x)  # ++x non
      è un incremento
```

Output:

```
Dopo x += 1: 6
Dopo x -= 1: 5
++x vale 5 e x vale ancora 5
```

Operatori relazionali

Operatori relazionali, usati per confrontare valori e restituire un risultato booleano:

- == (Uguale a) $\rightsquigarrow$ Verifica se due valori sono uguali
- != (Diverso da) $\rightsquigarrow$ Verifica se due valori sono diversi
- < (Minore di) $\rightsquigarrow$ Verifica se un valore è minore di un altro
- > (Maggiore di) $\rightsquigarrow$ Verifica se un valore è maggiore di un altro
- <= (Minore o uguale a) $\rightsquigarrow$ Verifica se un valore è minore o uguale a un altro
- >= (Maggiore o uguale a) $\rightsquigarrow$ Verifica se un valore è maggiore o uguale a un altro

Risultati booleani e confronti concatenati

- Gli operatori relazionali restituiscono un risultato booleano: `True` se la condizione è soddisfatta, `False` altrimenti
  - visualizzati come `True` e `False`, non come `1` e `0` come in C++
- Questi operatori sono usati comunemente nelle **istruzioni condizionali** e nei **cicli**
- I confronti possono essere concatenati: `0 < x <= 10` significa `0 < x and x <= 10`

Esempio: operatori relazionali (1/2)

```
1  a = 10
2  b = 5
3
4  print("a == b:", a == b)  # Uguale a
5  print("a != b:", a != b)  # Diverso da
6  print("a < b:", a < b)    # Minore di
7  print("a > b:", a > b)    # Maggiore di
8  print("a <= b:", a <= b)  # Minore o uguale a
9  print("a >= b:", a >= b)  # Maggiore o uguale a
10 print("0 < b <= a:", 0 < b <= a)  # Confronto
       concatenato
```

Esempio: operatori relazionali (2/2)

Output:

```
a == b: False
a != b: True
a < b: False
a > b: True
a <= b: False
a >= b: True
0 < b <= a: True
```

Operatori logici

Operatori logici, usati per le operazioni logiche, spesso nelle istruzioni condizionali:

- `and` (AND logico) $\rightsquigarrow$ Restituisce `True` se entrambe le condizioni sono `True`
- `or` (OR logico) $\rightsquigarrow$ Restituisce `True` se almeno una condizione è `True`
- `not` (NOT logico) $\rightsquigarrow$ Inverte il valore booleano

- Gli operatori logici di Python sono **parole**, mentre in C++ sono i simboli `&&`, `||` e `!`

Esempio: operatori logici (1/2)

```
1  condition1 = True
2  condition2 = False
3
4  print("condition1:", condition1)
5  print("condition2:", condition2)
6
7  # AND logico: True solo se entrambe le condizioni sono
       True
8  print("condition1 and condition2:", condition1 and
       condition2)
9
10 # OR logico: True se almeno una condizione è True
11 print("condition1 or condition2:", condition1 or
       condition2)
12
13 # NOT logico: inverte il valore booleano
14 print("not condition1:", not condition1)
15 print("not condition2:", not condition2)
```

Esempio: operatori logici (2/2)

Output:

```
condition1: True
condition2: False
condition1 and condition2: False
condition1 or condition2: True
not condition1: False
not condition2: True
```

Riepilogo (1/2)

Operatori aritmetici:

- `/` dà sempre un `float`; `//` è la divisione intera e arrotonda per difetto, anche per i numeri negativi; `**` è l'elevamento a potenza
- Conversione di tipo con `int()`, `float()`, `str()`

Assegnamento, incremento e decremento:

- Assegnamenti composti `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=`
- Niente `++` e `--`: si usano `x += 1` e `x -= 1`

Riepilogo (2/2)

Operatori relazionali:

- ==, !=, <, >, <=, >=, con risultati `True` e `False`
- I confronti possono essere concatenati: `0 < x <= 10`

Operatori logici:

- Le parole `and`, `or`, `not` al posto di `&&`, `||`, `!`

Istruzioni condizionali

Introduzione: Istruzioni condizionali

- Gli operatori relazionali e logici producono le condizioni che decidono quali istruzioni vengono eseguite
- In questa sezione vediamo le istruzioni `if`, `elif` ed `else`
- Vediamo anche l'espressione condizionale, la versione Python dell'operatore ternario di C++

Le istruzioni `if`, `elif` ed `else`

- Le istruzioni condizionali permettono di eseguire blocchi di codice diversi in base a condizioni specifiche
- L'istruzione `if` valuta una condizione. Se la condizione è `True`, il blocco indentato viene eseguito
- La clausola opzionale `elif` (else if) verifica una condizione aggiuntiva quando le precedenti sono `False`
- La clausola opzionale `else` viene eseguita quando tutte le condizioni precedenti sono `False`
- **Non servono parentesi** attorno alla condizione, e **non si usano parentesi graffe**: la riga della condizione termina con ":" e il blocco è delimitato dall'indentazione

Esempio: istruzione condizionale

```
1 number = 10
2
3 # Verifica se il numero è positivo
4 if number > 0:
5     print("Il numero è positivo.")
6 else:
7     print("Il numero non è positivo.")
```

Output:

```
Il numero è positivo.
```

Esempio: if / elif / else (1/2)

```
1  score = 85
2
3  if score >= 90:
4      grade = "A"
5  elif score >= 80:
6      grade = "B"
7  elif score >= 70:
8      grade = "C"
9  elif score >= 60:
10     grade = "D"
11 else:
12     grade = "F"
13
14 print(f"Punteggio: {score}, Voto: {grade}")
```

Esempio: `if` / `elif` / `else` (2/2)

Output:

```
Punteggio: 85, Voto: B
```

- Le condizioni sono valutate dall'alto verso il basso, e viene eseguito solo il blocco della prima condizione vera
- `score >= 90` è falsa e `score >= 80` è vera, quindi il voto è `"B"`
- Il risultato è lo stesso degli operatori ternari annidati del capitolo «Introduzione a C++»

Espressione condizionale

- L'**espressione condizionale** è la versione Python dell'operatore ternario `?:` di C++: offre un modo compatto per scrivere semplici istruzioni `if-else`
- Sintassi:

```
valore_se_vero if condizione else valore_se_falso
```

- Se la condizione è `True`, l'espressione vale `valore_se_vero`; altrimenti vale `valore_se_falso`

Esempio: espressione condizionale (1/3)

```python
number = 10

# Espressione condizionale compatta
result = "positivo" if number > 0 else "non positivo"
print(f"Il numero è {result}")

# Ricerca del massimo fra due numeri
a, b = 15, 20  # Assegnamento multiplo: a = 15 e b = 20
maximum = a if a > b else b
print(f"Massimo: {maximum}")
```

Esempio: espressione condizionale (2/3)

```
11
12 # Verifica se pari o dispari
13 n = 7
14 parity = "pari" if n % 2 == 0 else "dispari"
15 print(f"{n} è {parity}")
16
17 # Valore assoluto
18 x = -5
19 abs_value = x if x >= 0 else -x
20 print(f"Il valore assoluto di {x} è {abs_value}")
```

Esempio: espressione condizionale (3/3)

Output:

```
Il numero è positivo
Massimo: 20
7 è dispari
Il valore assoluto di -5 è 5
```

La libreria di Python fornisce anche le funzioni `max(a, b)` e `abs(x)`, che restituiscono gli stessi valori.

Riepilogo

Istruzioni condizionali:

- `if`, poi un numero qualunque di `elif`, poi un `else` opzionale
- Niente parentesi, niente parentesi graffe: "`:`" e indentazione
- Viene eseguito solo il blocco della prima condizione vera

Espressione condizionale:

- `x if c else y`, la versione Python di `c ? x : y`

Cicli

Introduzione: Cicli

- Le istruzioni condizionali scelgono *se* un blocco di codice viene eseguito; i cicli decidono *quante volte* viene eseguito
- I cicli servono a eseguire ripetutamente un blocco di codice
- In Python i due costrutti di ciclo sono il ciclo `while` e il ciclo `for`
- Vediamo anche le istruzioni `break` e `continue`

Il ciclo `while`

- Il ciclo `while` esegue un blocco di codice finché la condizione vale `True`
- Se la condizione è `False` fin dall'inizio, il blocco non viene mai eseguito

Esempio: ciclo `while`

```python
i = 1                   # Inizializzazione

while i <= 5:           # Condizione
    print(f"Numero: {i}")
    i += 1              # Incremento (in Python non esiste l'operatore ++)
```

Output:

```
Numero: 1
Numero: 2
Numero: 3
Numero: 4
Numero: 5
```

Come in C++, la condizione `i <= 5` è valutata sei volte: cinque volte è vera, e la sesta volta (`i` uguale a 6) è falsa e il ciclo termina.

Il ciclo `for`

- Il ciclo `for` di Python scorre gli elementi di un **iterabile**, come una lista, una stringa o un intervallo di interi: a ogni iterazione la variabile del ciclo assume l'elemento successivo
- La funzione `range(start, stop, step)` genera la sequenza di interi da `start` (incluso) a `stop` (escluso), con incremento `step`
  - se è dato un solo argomento, la sequenza parte da 0
  - se `step` è omesso, vale 1
  - con uno `step` negativo la sequenza decresce

Dallo pseudocodice a Python

- Il ciclo dello pseudocodice

  “**for** $i := 1$ **to** n”

  corrisponde a `for i in range(1, n + 1)`

- Il ciclo sugli indici di una lista di dimensione n è

  `for i in range(n)`

  con $i = 0, 1, \ldots, n - 1$

Esempio: ciclo `for` con `range`

```
for i in range(1, 6):   # i assume i valori 1, 2, 3, 4, 5
    print(f"Numero: {i}")
```

L'output è lo stesso del ciclo `while` dell'esempio precedente. La funzione `list()` mostra gli interi generati da un `range`:

```
print(list(range(5)))
print(list(range(1, 6)))
print(list(range(0, 10, 3)))
print(list(range(5, 0, -1)))
```

Output:

```
[0, 1, 2, 3, 4]
[1, 2, 3, 4, 5]
[0, 3, 6, 9]
[5, 4, 3, 2, 1]
```

Esempio: ciclo `for` su una lista

```
1 a = [3, 7, 2, 9, 5]
2
3 for element in a:
4     print(element)
```

Output:

```
3
7
2
9
5
```

Il ciclo visita direttamente gli elementi, senza usare i loro indici.

Le istruzioni `break` e `continue`

- L'istruzione `break` termina immediatamente il ciclo
- L'istruzione `continue` salta il resto dell'iterazione corrente e passa alla successiva

Esempio: break e continue

```python
for i in range(1, 11):
    if i % 2 == 0:
        continue  # Salta i numeri pari
    if i > 7:
        break     # Interrompe il ciclo al primo dispari maggiore di 7
    print(i, end=" ")
print()
```

Output:

```
1 3 5 7
```

- Come nella versione C++, i valori pari di `i` sono saltati da `continue`
- Quando `i` è uguale a 9 l'istruzione `break` termina il ciclo, e il valore 10 non viene mai raggiunto
- L'ultima `print()` scrive solo l'a capo

Riepilogo

Il ciclo `while`:

- Ripete il blocco finché la condizione vale `True`

Il ciclo `for`:

- Scorre gli elementi di un iterabile: un `range`, una lista, una stringa
- `range(start, stop, step)`: `stop` è escluso
- "**for** $i := 1$ **to** n" diventa `for i in range(1, n + 1)`

`break` **e** `continue`:

- Terminano il ciclo, o passano all'iterazione successiva

Liste

Introduzione: Liste

- I cicli diventano davvero utili quando visitano collezioni di valori
- In Python una collezione di valori è memorizzata in una **lista**
- Le liste svolgono il ruolo degli array di C++, e sono la struttura dati degli algoritmi che scriveremo
- Vediamo anche le liste bidimensionali, e un errore frequente da evitare

Liste (1/2)

- Una **lista** è una collezione ordinata e modificabile di elementi, che può contenere valori di qualunque tipo
  - le liste sono la principale struttura dati sequenziale di Python e svolgono il ruolo degli array di C++

- Le liste sono racchiuse fra parentesi quadre `[]` e gli elementi sono separati da virgole

- Gli elementi si raggiungono tramite il loro **indice**, a partire da `0`: gli elementi di una lista `a` di dimensione n sono `a[0]`, `a[1]`, ..., `a[n-1]`
  - la funzione `len(a)` restituisce la dimensione della lista
  - l'indice negativo `a[-1]` indica l'ultimo elemento

Liste (2/2)

- A differenza degli array di C++, le liste sono **dinamiche**: possono crescere o ridursi durante l'esecuzione senza gestire manualmente la memoria
  - l'istruzione `a.append(x)` aggiunge l'elemento `x` in fondo alla lista
- L'espressione `[0] * n` crea una lista di n zeri
- Più in generale, una **list comprehension** `[espressione for i in range(n)]` crea la lista degli n valori dell'espressione per $i = 0, 1, \ldots, n-1$
  - quando la variabile del ciclo non è usata, per convenzione si chiama `_`

Creare le liste

Le liste si creano con []
e si indicizzano a partire da 0.

Non servono né l'allocazione né la deallocazione della memoria.

Esempio: creazione e accesso a una lista

```
1  n = 5
2
3  # Crea una lista di n zeri
4  a = [0] * n
5
6  # Assegna i valori
7  for i in range(n):
8      a[i] = i * 2
9
10 print(a)        # L'intera lista
11 print(a[0])     # Primo elemento
12 print(a[4])     # Ultimo elemento, indice n-1
13 print(len(a))   # Numero di elementi
```

Output:

```
[0, 2, 4, 6, 8]
0
8
5
```

Esempio: lista scritta per esteso, modifica e crescita (1/2)

```python
b = [9, 12, 3, 16]   # Lista di 4 interi

b[2] = 5             # Assegna un nuovo valore al terzo elemento
b.append(20)         # La lista cresce: ora ha 5 elementi
print(b, len(b))
print(b[-1])         # L'ultimo elemento

squares = [i * i for i in range(6)]  # List comprehension
print(squares)
```

Esempio: lista scritta per esteso, modifica e crescita (2/2)

Output:

```
[9, 12, 5, 16, 20] 5
20
[0, 1, 4, 9, 16, 25]
```

Accedere a un indice che non esiste, per esempio `b[5]`, ferma il programma con l'errore `IndexError: list index out of range` (in C++ lo stesso accesso non è controllato).

Esempio: lista scritta per esteso e iterazione sugli indici

```
1 a = [9, 12, 3, 16, 19, 11, 12]
2 n = len(a)
3
4 for i in range(n):
5     print(f"a[{i}] = {a[i]}")
```

Output:

```
a[0] = 9
a[1] = 12
a[2] = 3
a[3] = 16
a[4] = 19
a[5] = 11
a[6] = 12
```

Liste bidimensionali

- I dati bidimensionali possono essere rappresentati con una **lista di liste**
- Ogni riga è a sua volta una lista, e gli elementi si raggiungono con due indici `matrix[i][j]`, entrambi a partire da 0
  - per esempio, se `c = [[3, 1, 4], [1, 5, 9]]`, allora `c[1][2]` è l'elemento nella seconda riga e terza colonna, uguale a 9
- Una matrice di m righe e n colonne riempita di zeri si crea con la list comprehension `[[0] * n for _ in range(m)]`, che costruisce m righe distinte

Esempio: lista bidimensionale (matrice) (1/2)

```python
m = 3   # Numero di righe
n = 2   # Numero di colonne

# Crea una matrice di zeri
matrix = [[0] * n for _ in range(m)]

# Assegna i valori
for i in range(m):
    for j in range(n):
        matrix[i][j] = i * j

# Stampa la matrice
for i in range(m):
    for j in range(n):
        print(f"matrix[{i}][{j}] = {matrix[i][j]}")
print(matrix)
```

Esempio: lista bidimensionale (matrice) (2/2)

Output:

```
matrix[0][0] = 0
matrix[0][1] = 0
matrix[1][0] = 0
matrix[1][1] = 1
matrix[2][0] = 0
matrix[2][1] = 2
[[0, 0], [0, 1], [0, 2]]
```

Esempio: un errore frequente con le liste bidimensionali

- L'espressione più breve `[[0] * n] * m` è sbagliata: ripete *m* volte la *stessa* riga, quindi una modifica a una riga compare in tutte le righe

```
1 wrong = [[0] * 2] * 3                    # Tre
      riferimenti alla stessa riga
2 wrong[0][0] = 1
3 print(wrong)
4
5 right = [[0] * 2 for _ in range(3)]      # Tre righe
      distinte
6 right[0][0] = 1
7 print(right)
```

Output:

```
[[1, 0], [1, 0], [1, 0]]
[[1, 0], [0, 0], [0, 0]]
```

Riepilogo

Liste:

- Collezioni ordinate, modificabili e dinamiche, scritte con `[]`
- Indici da `0` a `len(a)-1`; `a[-1]` è l'ultimo elemento
- `append` aggiunge un elemento in fondo; `[0] * n` e le list comprehension creano liste
- Un indice fuori dall'intervallo dà un `IndexError`

Liste bidimensionali:

- Liste di liste, a cui si accede con `matrix[i][j]`
- Si creano con `[[0] * n for _ in range(m)]`, mai con `[[0] * n] * m`

Funzioni

Introduzione: Funzioni

- Come nello pseudocodice, un algoritmo è scritto come una funzione che riceve l'input e restituisce l'output
- In questa sezione vediamo come si definisce e come si chiama una funzione in Python
- Vediamo anche che cosa può fare una funzione alle variabili di chi la chiama, e come restituisce più valori

Definire una funzione

- Le funzioni permettono di scrivere codice modulare e riutilizzabile, raggruppando le istruzioni in una sola unità
- Una funzione si definisce con la parola chiave `def`, seguita dal nome della funzione e dall'elenco dei parametri fra parentesi, e termina con il simbolo ":"

Le parti di una funzione

Una funzione in Python è composta da:

1. la parola chiave `def` seguita dal **nome della funzione**
2. un **elenco di parametri** (opzionale): specifica i valori in ingresso, senza i loro tipi di dato
3. un **corpo** indentato: il blocco di codice eseguito quando la funzione viene chiamata
4. un'istruzione `return` (opzionale): restituisce un valore a chi chiama la funzione
   - una funzione senza `return` restituisce il valore speciale `None`

Dove si scrive il codice (1/2)

- Una funzione deve essere definita prima che venga eseguita l'istruzione che la chiama: per questo motivo le funzioni sono scritte all'inizio del file

- Le istruzioni principali di un programma Python sono tipicamente poste al livello più esterno del file, oppure dentro un blocco protetto da

  ```
  if __name__ == "__main__":
  ```

  che garantisce che il codice venga eseguito solo quando il file è eseguito direttamente (e non quando è importato come modulo da un altro file)

Dove si scrive il codice (2/2)

- Questo blocco svolge il ruolo della funzione `main` di C++
- Il codice sorgente Python può anche essere eseguito online, per esempio al seguente indirizzo: https://www.mycompiler.io/online-python-compiler

Esempio: funzioni (1/2)

```python
# Definizione di funzione
def addition(a, b):
    return a + b  # Restituisce la somma

# Definizione di funzione
def multiplication(a, b):
    return a * b  # Restituisce il prodotto

if __name__ == "__main__":
    total = addition(5, 10)          # Chiamata di funzione
    print(f"Somma: {total}")

    product = multiplication(5, 10)  # Chiamata di funzione
    print(f"Prodotto: {product}")
```

Esempio: funzioni (2/2)

Output:

```
Somma: 15
Prodotto: 50
```

Esempio: funzione su una lista bidimensionale (1/2)

```python
def sum_matrix(matrix, rows, columns):
    """Restituisce la somma di tutti gli elementi di una lista 2D."""
    total = 0
    for i in range(rows):
        for j in range(columns):
            total += matrix[i][j]
    return total

if __name__ == "__main__":
    m, n = 3, 2

    # Costruisce la matrice con una list comprehension annidata
    matrix = [[i * j for j in range(n)] for i in range(m)]

    result = sum_matrix(matrix, m, n)
    print(f"Somma degli elementi della matrice: {result}")
```

Esempio: funzione su una lista bidimensionale (2/2)

Output:

```
Somma degli elementi della matrice: 3
```

- La matrice è `[[0, 0], [0, 1], [0, 2]]`, come nell'esempio della lista bidimensionale (matrice)
- La somma dei suoi elementi è $0 + 0 + 0 + 1 + 0 + 2 = 3$

Parametri e valori di ritorno

- In C++ i parametri sono passati per valore o per riferimento
- In Python ogni parametro riceve lo *stesso* oggetto che è l'argomento di chi chiama
- L'effetto di una funzione su chi la chiama dipende da ciò che la funzione fa con esso

Che cosa fa una funzione con i suoi parametri

- Se la funzione **assegna un nuovo valore** a un parametro (per esempio `x = x + 1`), cambia solo il nome locale: la variabile di chi chiama non è toccata
  - per i numeri, il comportamento è lo stesso dei parametri passati per valore in C++
- Se la funzione **modifica gli elementi** di una lista ricevuta come parametro (per esempio `a[i] = 0`), la modifica è visibile a chi chiama, come per gli array passati a una funzione in C++
- Una funzione può restituire **più valori** contemporaneamente, separati da virgole
  - sono restituiti come una **tupla**, e chi chiama li assegna a più variabili con un assegnamento multiplo
  - in C++ lo stesso risultato richiede parametri passati per riferimento

Esempio: parametri e valori di ritorno multipli (1/3)

```
1 def increment(x):
2     x = x + 1                # Cambia solo il nome locale x
3
4 def fill_with_zeros(a):
5     for i in range(len(a)):
6         a[i] = 0             # Modifica la lista di chi
      chiama
7
8 def divide(n, m):
9     return n // m, n % m  # Restituisce due valori (una
       tupla)
```

Esempio: parametri e valori di ritorno multipli (2/3)

```
11 if __name__ == "__main__":
12     k = 5
13     increment(k)
14     print("Dopo increment:", k)
15 
16     b = [4, 7, 1]
17     fill_with_zeros(b)
18     print("Lista:", b)
19 
20     q, r = divide(17, 5)
21     print("Quoziente e resto:", q, r)
22     print("Tupla:", divide(17, 5))
```

Esempio: parametri e valori di ritorno multipli (3/3)

Output:

```
Dopo increment: 5
Lista: [0, 0, 0]
Quoziente e resto: 3 2
Tupla: (3, 2)
```

Riepilogo

Funzioni:

- `def nome(parametri):`, un corpo indentato, un `return` opzionale (altrimenti `None`)
- Definite prima di essere chiamate, all'inizio del file
- Il blocco `if __name__ == "__main__":` svolge il ruolo di `main`

Parametri e valori di ritorno:

- Assegnare a un parametro non cambia chi chiama; modificare gli elementi di una lista sì
- Più valori sono restituiti insieme come una tupla

Un algoritmo completo: il valore massimo in un array

Introduzione: Un algoritmo completo

- Mettiamo ora insieme tutti i pezzi: una funzione, un ciclo, un'istruzione condizionale e una lista
- Traduciamo un algoritmo del corso dallo pseudocodice a Python, facendo attenzione agli indici
- Vediamo anche come generare a caso un'istanza di prova, ed eseguiamo l'algoritmo passo per passo

Il problema

Consideriamo il problema di trovare
il valore massimo in un array a
di $n \geq 1$ numeri interi.

- La slide seguente mostra lo pseudocodice dell'algoritmo `FindMaximum` del capitolo «Pseudocodice»: le componenti sono indicizzate da 1 a n

Lo pseudocodice

Algorithm: FindMaximum

Input : Un array $a[1..n]$ di $n \geq 1$ numeri interi
Output: Il valore massimo nell'array a

```
1 function FindMaximum(a, n)
2 |   max := a[1]
3 |   for i := 2 to n do
4 |   |   if a[i] > max then
5 |   |   |   max ← a[i]
6 |   |   end
7 |   end
8 |   return max
9 end
```

Dallo pseudocodice a Python

- Gli elementi sono indicizzati da 0 a $n-1$, quindi l'inizializzazione usa `a[0]` e il ciclo `for i in range(1, len(a))` visita gli indici $1, \ldots, n-1$
- La dimensione n non è passata come parametro, perché la funzione la ottiene con `len(a)`
- La variabile si chiama `max_val` e non `max`, per non nascondere la funzione predefinita `max()`
- La funzione suppone $n \geq 1$, come lo pseudocodice: per una lista vuota l'accesso ad `a[0]` ferma il programma con un `IndexError`

L'implementazione in Python (1/2)

```python
def array_max_value(a):
    """Restituisce il valore massimo della lista a (len(a) >= 1)."""
    max_val = a[0]  # Inizializza con il primo elemento

    for i in range(1, len(a)):
        if a[i] > max_val:
            max_val = a[i]  # Aggiorna il massimo

    return max_val

if __name__ == "__main__":
    a = [9, 12, 3, 16, 19, 11, 12]

    max_val = array_max_value(a)
    print(f"Valore massimo: {max_val}")
```

L'implementazione in Python (2/2)

Output:

```
Valore massimo: 19
```

Un'istanza di prova casuale

- L'istanza di prova può anche essere generata a caso con il modulo `random` della libreria di Python
- `random.randint(1, 100)` restituisce un intero casuale fra 1 e 100 (entrambi inclusi)
- L'istruzione `random.seed(2026)` fissa il punto di partenza del generatore casuale, in modo che ogni esecuzione produca la stessa lista
- Senza di essa, ogni esecuzione produce una lista diversa

Esempio: un'istanza casuale (1/2)

```python
import random

def array_max_value(a):
    max_val = a[0]
    for i in range(1, len(a)):
        if a[i] > max_val:
            max_val = a[i]
    return max_val

if __name__ == "__main__":
    random.seed(2026)  # Stessa lista casuale a ogni esecuzione
    n = 10             # Dimensione della lista
    a = [random.randint(1, 100) for _ in range(n)]

    print(a)
    print(f"Valore massimo: {array_max_value(a)}")
```

Esempio: un'istanza casuale (2/2)

Output:

```
[16, 41, 65, 66, 83, 14, 29, 77, 80, 72]
Valore massimo: 83
```

Esempio: esecuzione passo per passo (1/3)

- Eseguiamo la funzione `array_max_value` su due istanze
- In ogni tabella, la colonna i è l'indice del ciclo Python e la colonna $i + 1$ è l'indice corrispondente dello pseudocodice
- La prima riga è l'inizializzazione `max_val = a[0]`

Esempio: esecuzione passo per passo (2/3)

Istanza 1: $a = [9, 12, 3, 16, 19, 11, 12]$, $n = 7$

i (Python)	$i+1$ (pseudocodice)	`a[i]`	`a[i]` > `max_val`?	`max_val`
–	–	–	–	9
1	2	12	Sì	12
2	3	3	No	12
3	4	16	Sì	16
4	5	19	Sì	19
5	6	11	No	19
6	7	12	No	19

Restituisce: `max_val` $= 19$

Esempio: esecuzione passo per passo (3/3)

Istanza 2: $a = [3, 7, 2, 9, 1]$, $n = 5$

i (Python)	$i+1$ (pseudocodice)	`a[i]`	`a[i]` > `max_val`?	`max_val`
–	–	–	–	3
1	2	7	Sì	7
2	3	2	No	7
3	4	9	Sì	9
4	5	1	No	9

Restituisce: `max_val` $= 9$

Riepilogo

- Lo pseudocodice `FindMaximum` diventa la funzione `array_max_value`
- $a[1]$ diventa `a[0]`, e "**for** $i := 2$ **to** n" diventa `for i in range(1, len(a))`
- La dimensione si ottiene con `len(a)`; il nome `max_val` non nasconde la funzione predefinita `max()`
- Istanze casuali con `random.randint`; `random.seed` le rende riproducibili
- Le tabelle passo per passo associano a ogni indice i di Python l'indice $i + 1$ dello pseudocodice

Operatori bit a bit

Introduzione: Operatori bit a bit

- Gli operatori bit a bit del capitolo «Introduzione a C++» esistono in Python con gli stessi simboli e lo stesso significato
- In questa sezione li elenchiamo e vediamo l'unica differenza: il NOT bit a bit
- La differenza viene dagli interi di Python, che non hanno un numero fisso di bit

Gli operatori bit a bit

- `&` (AND)
- `|` (OR)
- `^` (XOR)
- `~` (NOT)
- `<<` (scorrimento a sinistra)
- `>>` (scorrimento a destra)

La funzione `bin()` restituisce la rappresentazione binaria di un intero, preceduta dal prefisso `0b`.

Il NOT bit a bit

- Poiché gli interi di Python non hanno un numero fisso di bit, il NOT bit a bit non dipende da un tipo di dato
- `~x` è sempre uguale a $-x-1$
- Per ottenere il risultato a 8 bit di C++, il risultato è combinato con la maschera `0xFF` (il numero 255, la cui rappresentazione binaria è 11111111)

Esempio: operatori bit a bit (1/2)

```
1  a = 12  # Binario: 1100
2  b = 10  # Binario: 1010
3
4  print(a & b, a | b, a ^ b)           # AND, OR, XOR
5  print(bin(a), bin(b), bin(a & b))    # Rappresentazioni
       binarie
6  print(5 << 2, 20 >> 2, 20 >> 3)      # Scorrimenti
7  print(~a, ~a & 0xFF)                 # NOT, e la sua
       versione a 8 bit
8
9  n = 15
10 print("dispari" if n & 1 else "pari")   # Bit meno
       significativo
```

Esempio: operatori bit a bit (2/2)

Output:

```
8 14 6
0b1100 0b1010 0b1000
20 5 2
-13 243
dispari
```

Tutti i valori coincidono con quelli degli esempi corrispondenti del capitolo «Introduzione a C++».

Riepilogo

- `&`, `|`, `^`, `~`, `<<`, `>>`: gli stessi simboli e lo stesso significato che in C++
- `bin()` dà la rappresentazione binaria, con il prefisso `0b`
- `~x` $= -x - 1$; il risultato a 8 bit di C++ è `~x & 0xFF`

Python e C++ a confronto

Introduzione: Python e C++ a confronto

- Abbiamo incontrato gli stessi costrutti in due linguaggi
- La tabella seguente riassume le principali differenze fra i due linguaggi incontrate in questo capitolo
- È un riferimento rapido da usare quando si passa da un linguaggio all'altro

Python e C++ (1/3)

	C++	Python
Esecuzione	compilato (g++), poi eseguito	interpretato (`python3`)
Tipi di dato	dichiarati: `int x = 5;`	non dichiarati: `x = 5`
Fine dell'istruzione	`;`	fine della riga
Blocchi	parentesi graffe `{}`	`:` e indentazione
Commenti	`//` e `/* */`	`#` (e `""" """`)
Valori booleani	`true`, `false` (stampati `1`, `0`)	`True`, `False`

Python e C++ (2/3)

	C++	Python
`10 / 3`	3 (divisione intera)	`3.3333333333333335`
Divisione intera	`/` fra due `int`	`//`
Potenza	funzione `pow`	operatore `**`
Incremento	`x++`, `++x`	`x += 1`
Operatori logici	`&&`, `||`, `!`	`and`, `or`, `not`
Operatore ternario	`c ? x : y`	`x if c else y`

Python e C++ (3/3)

	C++	Python
Ciclo `for`	`for (int i = 0; i < n; i++)`	`for i in range(n):`
Collezioni	array, di dimensione fissa o dinamica	liste, dinamiche
Dimensione di una collezione	memorizzata in una variabile separata	`len(a)`
Più risultati	parametri per riferimento	restituzione di una tupla
Punto di ingresso	funzione `main`	livello più esterno del file

Riepilogo

- C++ è compilato e dichiara i tipi di dato; Python è interpretato e non li dichiara
- Python sostituisce “;” e le parentesi graffe con la fine della riga, “:” e l’indentazione
- Divisione, incremento, operatori logici e ternario si scrivono in modo diverso
- Le liste sono dinamiche e conoscono la propria dimensione; più risultati sono restituiti come una tupla

Riepilogo (1/2)

Punti chiave del capitolo:

- Un programma Python è eseguito dall'interprete (`python3`) dalla prima riga all'ultima; le istruzioni non terminano con "`;`" e i blocchi sono delimitati da "`:`" e dall'indentazione
- Le variabili sono create tramite assegnamento, senza dichiarare il loro tipo di dato; gli interi non hanno un valore massimo
- L'operatore `/` dà sempre un `float`; la divisione intera è `//` e arrotonda per difetto, anche per i numeri negativi

Riepilogo (2/2)

- Le istruzioni condizionali (`if`, `elif`, `else`, e l'espressione condizionale) scelgono quali istruzioni vengono eseguite; i cicli (`while`, e `for` su un `range` o su una lista) le ripetono
- Le liste di Python sono indicizzate da 0 a $n-1$, con $n =$ `len(a)`, mentre gli array dello pseudocodice sono indicizzati da 1 a n; le liste crescono con `append` e non richiedono gestione della memoria
- Una funzione modifica la lista di chi chiama quando ne cambia gli elementi, ma non un numero che riceve; più risultati sono restituiti insieme come una tupla